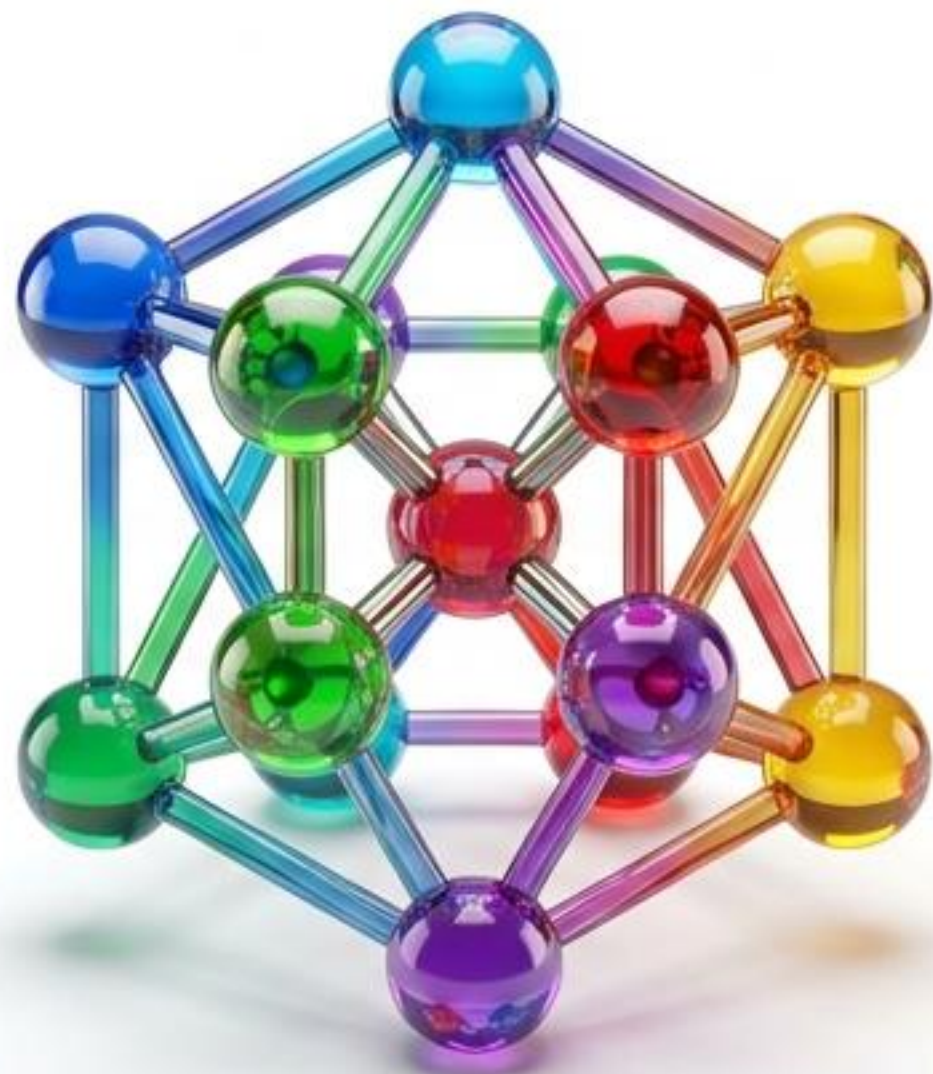




Học Sâu (Deep Learning)

Chương 2: Các khối xây dựng toán học của Mạng Nơ-ron

Đại học Đông Á
August 14, 2026

Nội dung chương

1. Ví dụ đầu tiên về
Mạng Nơ-ron (MNIST)



2. Biểu diễn dữ liệu bằng
Tensor



3. Phép toán Tensor
(Tensor Operations)



4. Tối ưu hóa Dựa trên
Gradient



5. Đồ thị Tính toán &
Lan truyền ngược



6. Tổng kết



Ví dụ kinh điển: Tập dữ liệu MNIST

Bài toán: Phân loại ảnh các chữ số viết tay (từ 0 đến 9).

Dữ liệu: 60.000 ảnh huấn luyện,
10.000 ảnh kiểm tra.

Kích thước mỗi ảnh: 28× 28 pixels (ảnh xám đen trắng).

Trong học máy:

Phân loại (Category) gọi là Lớp (Class).

Dữ liệu đầu vào gọi là Mẫu (Sample).

Kết quả của mẫu gọi là Nhãn (Label).



Hình 2.1: Một số mẫu ảnh từ MNIST



Tải dữ liệu MNIST trong Keras

Tập dữ liệu MNIST được tải sẵn trong Keras, dưới dạng 4 mảng NumPy.

```
from keras.datasets import mnist

(train_images , train_labels), (test_images , test_labels) = mnist.load_data ()
```

- Tập huấn luyện (Training set): train_images và train_labels
- Tập kiểm tra (Test set): test_images và test_labels
- Hình ảnh được mã hóa dưới dạng mảng NumPy, nhãn là các số từ 0 đến 9.

Khám phá hình dạng dữ liệu

Mảng NumPy là thư viện phổ biến để tính toán số. Nó được sử dụng làm định dạng lưu trữ trao đổi dữ liệu. Hãy xem xét cấu trúc của tập huấn luyện:

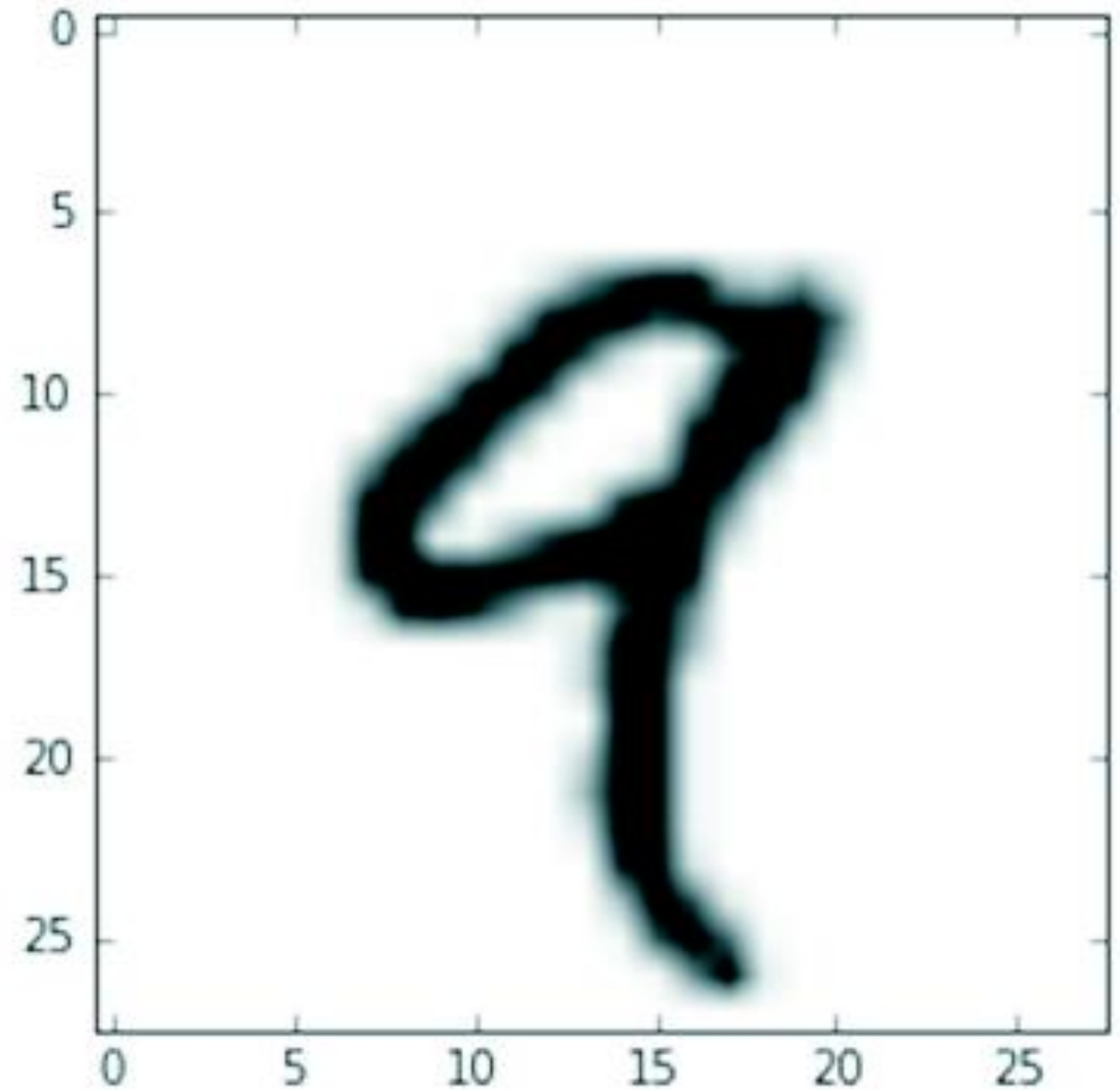
```
>>> train_images.shape  
(60000 , 28, 28)  
>>> len(train_labels)  
60000  
>>> test_images.shape  
(10000 , 28, 28)
```



Dữ liệu huấn luyện gồm 60.000 mẫu, mỗi mẫu là một ma trận 28× 28.

Xem xét một mẫu dữ liệu (Sample)

- Mỗi bức ảnh là một ma trận số nguyên.
- Giá trị từ 0 (trắng) đến 255 (đen).
- Ở đây chúng ta hiển thị mẫu thứ 4 trong tập huấn luyện (chữ số '9').



Hình 2.2: Biểu diễn ma trận của một ảnh



Kiến trúc của mô hình mạng nơ-ron

Khối xây dựng cốt lõi của mạng nơ-ron là **lớp (layer)**.

Các lớp **trích xuất các biểu diễn** từ dữ liệu được cung cấp.

Lớp **Dense** là lớp được kết nối đầy đủ (**fully connected**).

Lớp cuối cùng là **softmax 10 chiều**, trả về mảng 10 điểm xác suất.

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense (512, activation="relu"),
        layers.Dense (10, activation="softmax"),
    ]
)
```




Biên dịch mô hình (Compile)

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)
```



Tiền xử lý dữ liệu

Biến đổi ảnh 28× 28 thành vector 784 chiều, đưa giá trị về khoảng [0, 1].

```
train_images = train_images.reshape((60000 , 28 * 28))  
train_images = train_images.astype("float32") / 255
```


Huấn luyện (Training)



Quá trình huấn luyện (khớp dữ liệu) trong Keras được thực hiện thông qua **model.fit()**.



Đánh giá (Evaluation)



Sau khi huấn luyện, chúng ta đánh giá mô hình trên tập kiểm tra:

```
model.fit(train_images , train_labels , epochs=5,  
          batch_size=128)
```

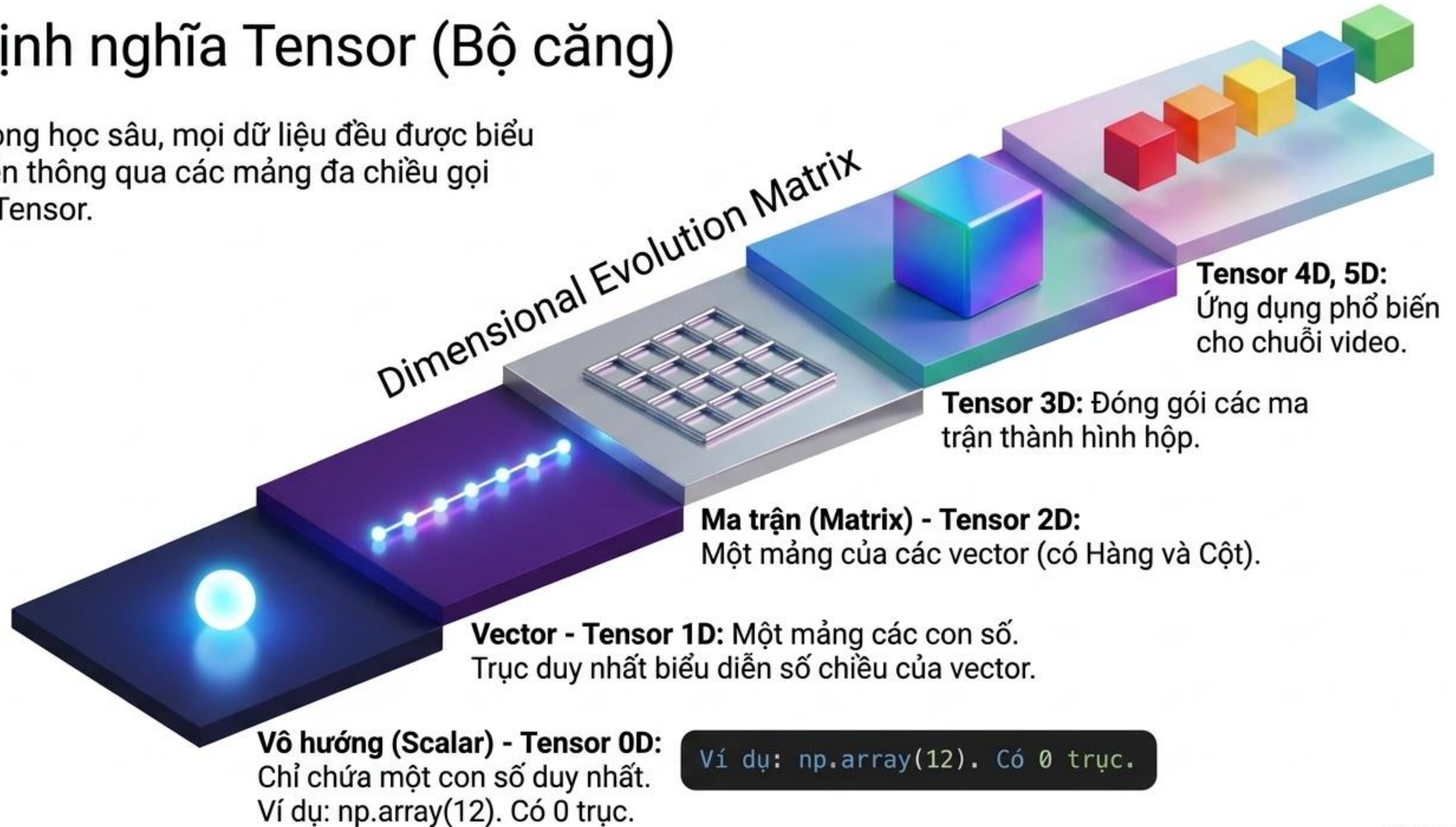
```
test_loss , test_acc =  
model.evaluate(test_images , test_labels)  
print(f"test_acc: {test_acc}") # Khoảng 97.8%
```



Hiện tượng mô hình hoạt động kém hơn trên dữ liệu mới so với dữ liệu huấn luyện gọi là **Overfitting (Quá khớp)**.

Định nghĩa Tensor (Bộ căng)

Trong học sâu, mọi dữ liệu đều được biểu diễn thông qua các mảng đa chiều gọi là Tensor.



Các thuộc tính chính của Tensor

Một tensor được xác định bởi ba thuộc tính cốt lõi:



1. Số trục (thứ hạng - rank):

Được biểu thị bằng ndim trong NumPy. Một ma trận có 2 trục, Tensor 3D có 3 trục.



2. Hình dạng (shape):

Là một tuple chứa số nguyên mô tả số chiều dọc theo mỗi trục. Ví dụ: (60000, 28, 28).



3. Kiểu dữ liệu (dtype):

Loại dữ liệu chứa trong tensor (float32, uint8, float64, bool v.v...).

Thao tác với Tensor: Cắt (Slicing) và Khái niệm Lô (Batch)



Cắt Tensor (Slicing): Lựa chọn phần tử cụ thể dọc theo các trục.

```
my_slice = train_images [10:100] # Chọn từ index 10 đến 99  
# Tương đương: train_images [10:100 , :, :]  
# Cắt ở giữa bức ảnh 14x14:  
my_slice = train_images[:, 7:-7, 7:-7]
```

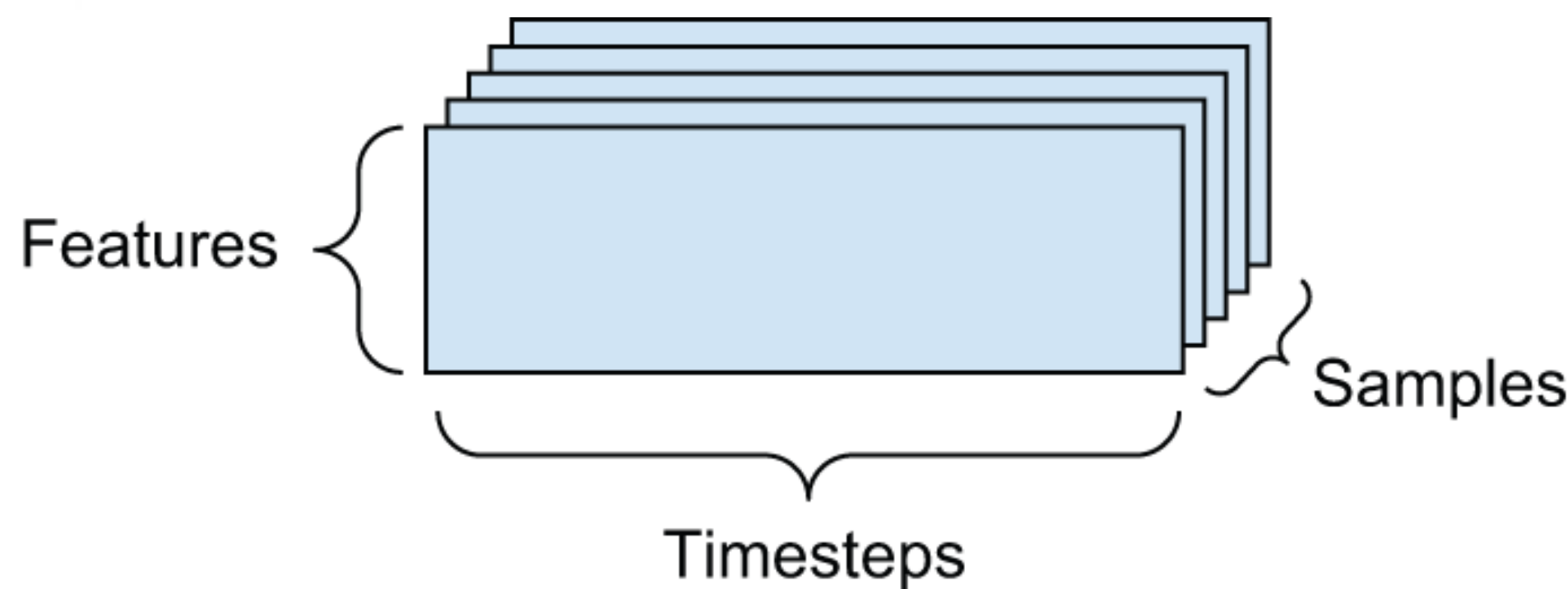


Khái niệm Lô (Batch): Mô hình học sâu xử lý dữ liệu theo các đợt nhỏ. Trục đầu tiên (trục 0) thường được gọi là trục lô (batch axis).

```
batch = train_images[:128] # Lô đầu tiên gồm 128 mẫu
```


Ví dụ thực tế: Vector và Dữ liệu Chuỗi thời gian

- Dữ liệu Chuỗi thời gian (Tensor 3D)
- Hình dạng: (mẫu, thời gian, đặc trưng)
- Trục thời gian luôn là trục thứ hai (index 1).
- Ví dụ: Giá cổ phiếu mỗi phút (hiện tại, cao nhất, thấp nhất) trong 390 phút, 250 ngày giao dịch → Tensor (250, 390, 3).



Hình 2.3: Tensor 3D cho dữ liệu chuỗi thời gian

Ví dụ thực tế: Dữ liệu Ảnh

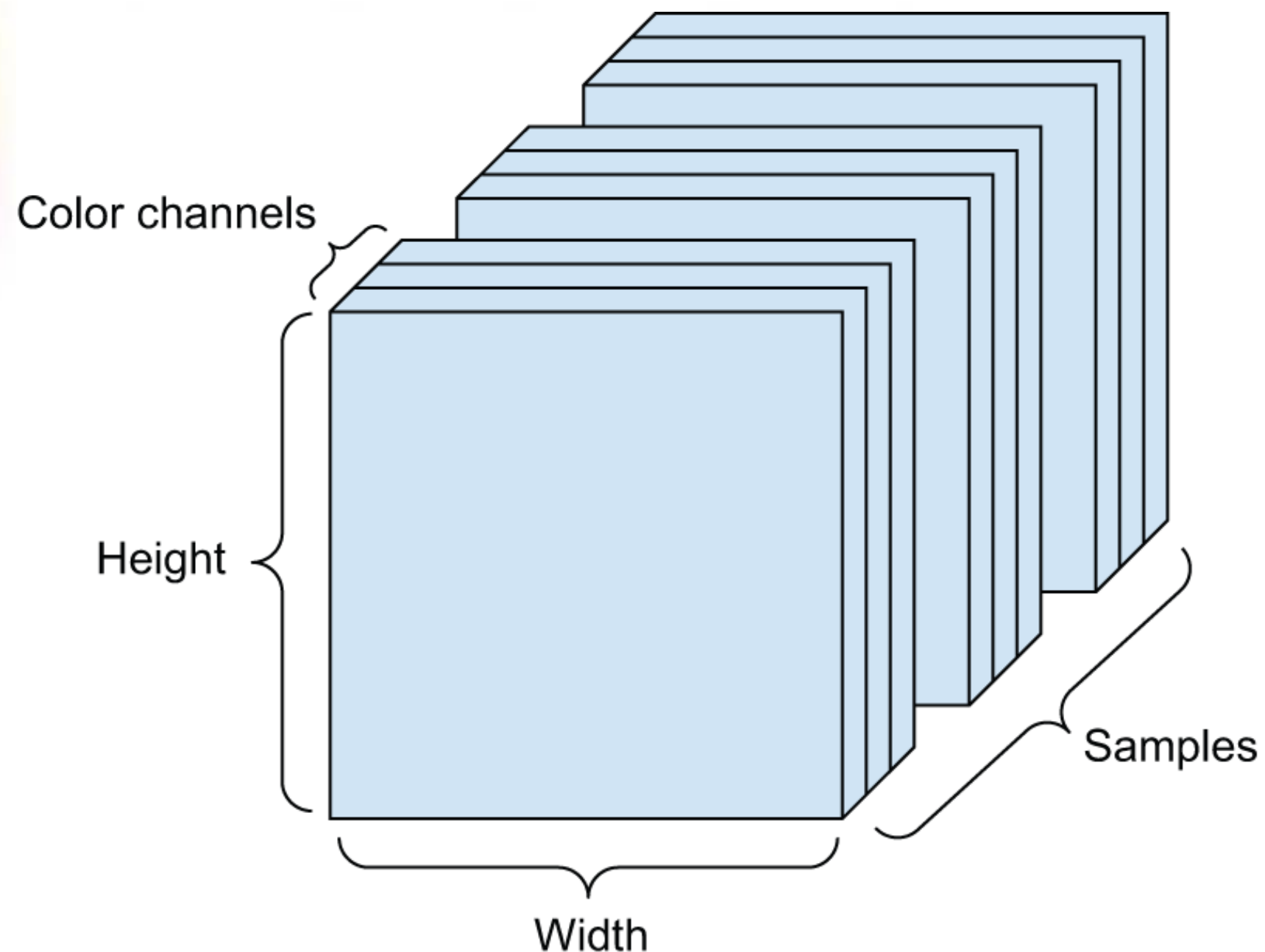


Ảnh kỹ thuật số thường có 3 chiều: Chiều cao, Chiều rộng và Kênh màu.

Lô ảnh (Batch of images): Tạo thành một Tensor 4D.

Hình dạng: (mẫu, chiều cao, chiều rộng, kênh màu)

Ví dụ: Lô 128 bức ảnh màu cỡ 256×256 có hình dạng là (128, 256, 256, 3).



Hình 2.4: Tensor 4D biểu diễn dữ liệu ảnh màu

Ví dụ thực tế: Dữ liệu Video



Dữ liệu video là trường hợp phức tạp nhất trong thực tế, sử dụng Tensor 5D.

Một video là một chuỗi các khung hình (frames), mỗi frame là ảnh màu 3D.

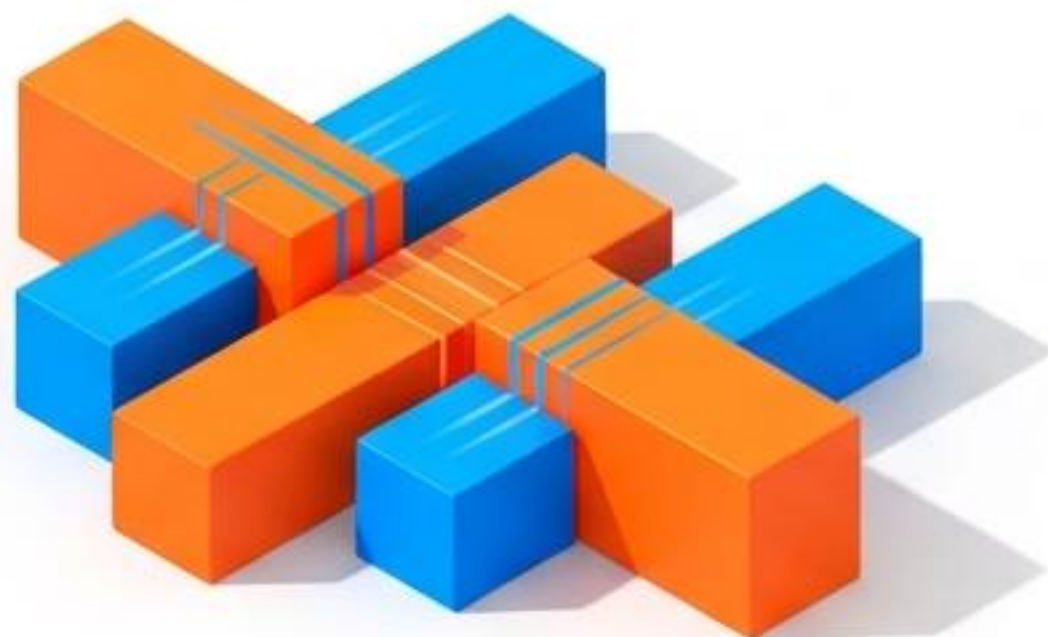
Hình dạng của tensor: (mẫu, khung hình, chiều cao, chiều rộng, kênh màu)

Ví dụ: Một lô 4 clip YouTube 60 giây (144×256 pixels, 4 fps) có 240 khung hình sẽ có hình dạng (4, 240, 144, 256, 3).

Kích thước này rất lớn
(tương đương ~ 425 MB
cho kiểu float32).



Phép toán Tensor: Xử lý theo phần tử (Element-wise)



Khái niệm

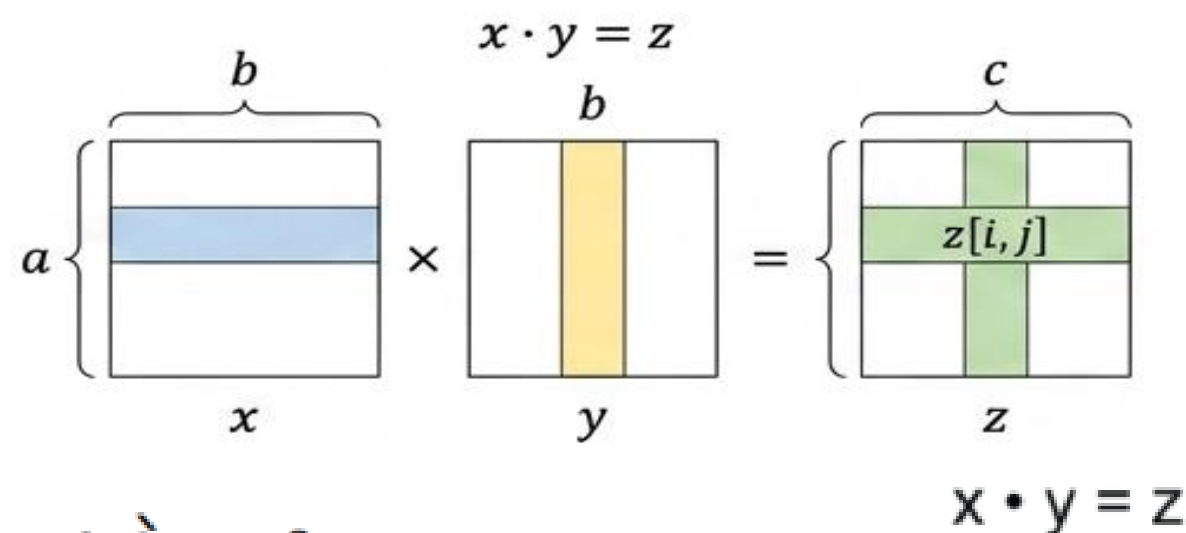
Áp dụng toán học độc lập cho từng mục bên trong tensor (Cộng, Trừ, Hàm ReLU...).



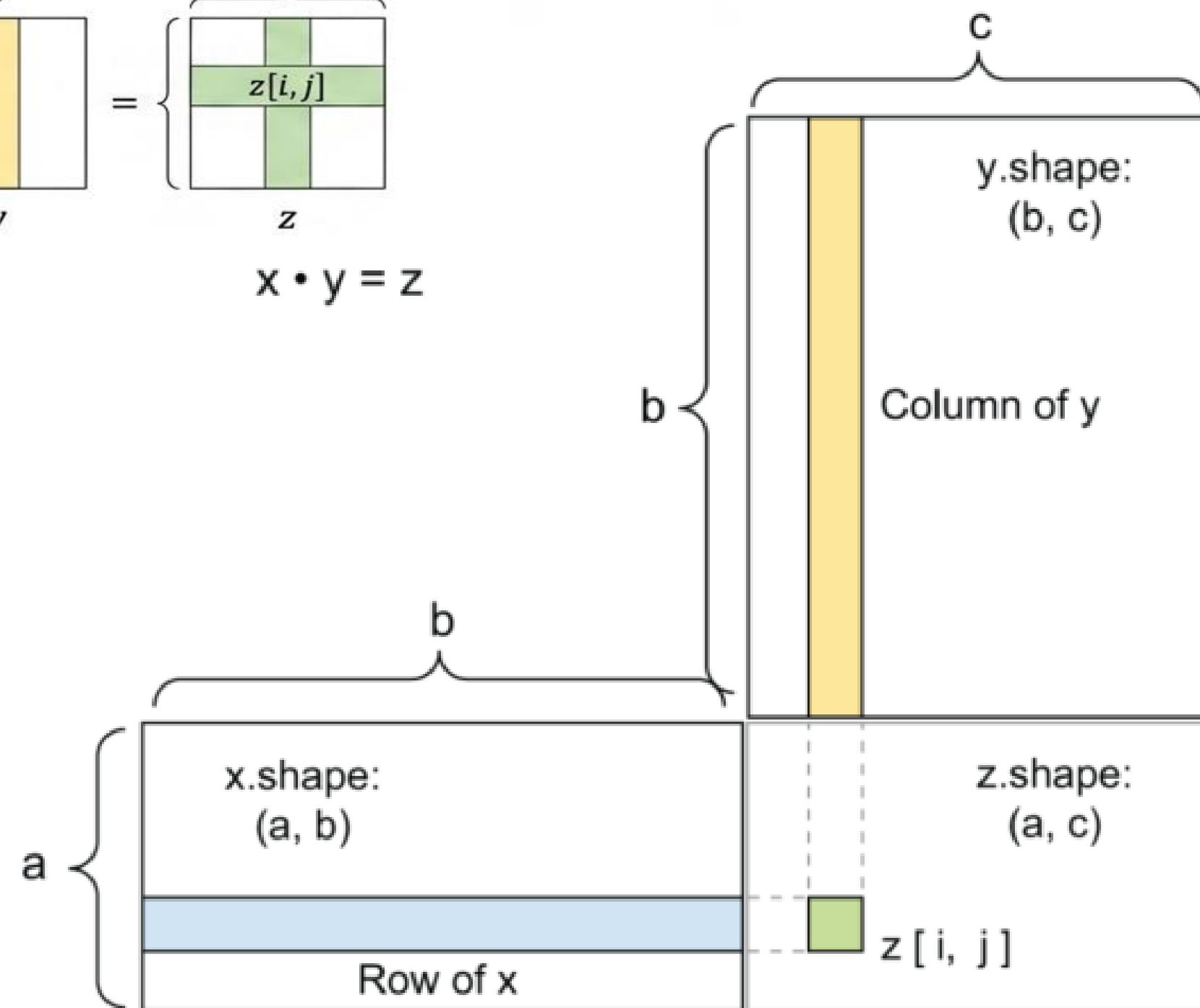
Hiệu suất

Tốc độ cực cao. Được thực thi song song toàn diện qua các thư viện BLAS siêu tốc trên kiến trúc GPU.

Sản phẩm Tensor (Tích vô hướng - Dot Product)



- Không giống phép nhân từng phần tử, **Tích Tensor** (`matmul` hoặc toán tử `@`) là phép toán phổ biến nhất.
- **Tích ma trận:** Kết hợp các hàng ma trận 1 với cột ma trận 2.
- Khả năng tương thích hình dạng: $(a, b) \cdot (b, c) \rightarrow (a, c)$

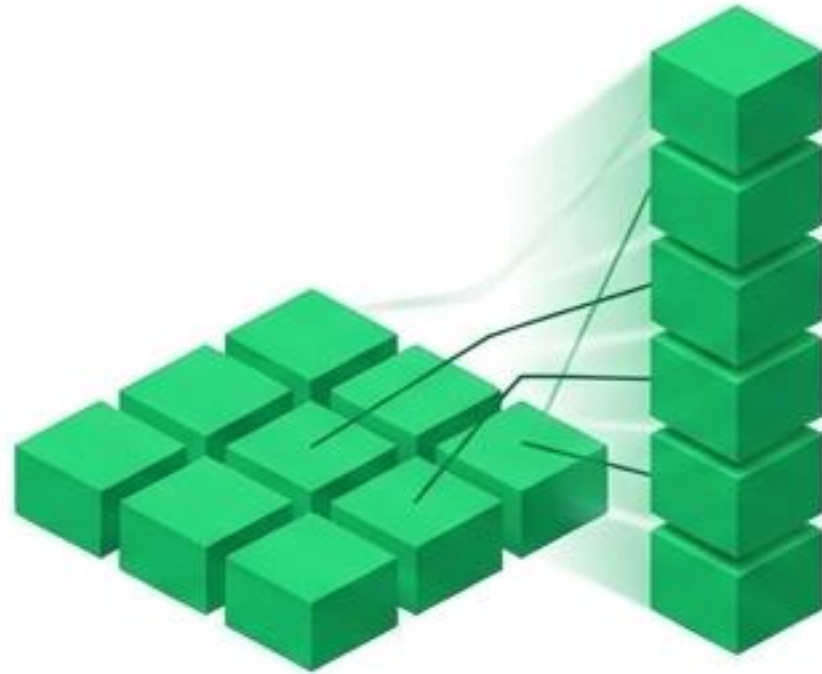


Hình 2.5: Sơ đồ hộp minh họa tích ma trận

Biến đổi Không gian: Reshape và Transpose

Định hình lại (Reshaping)

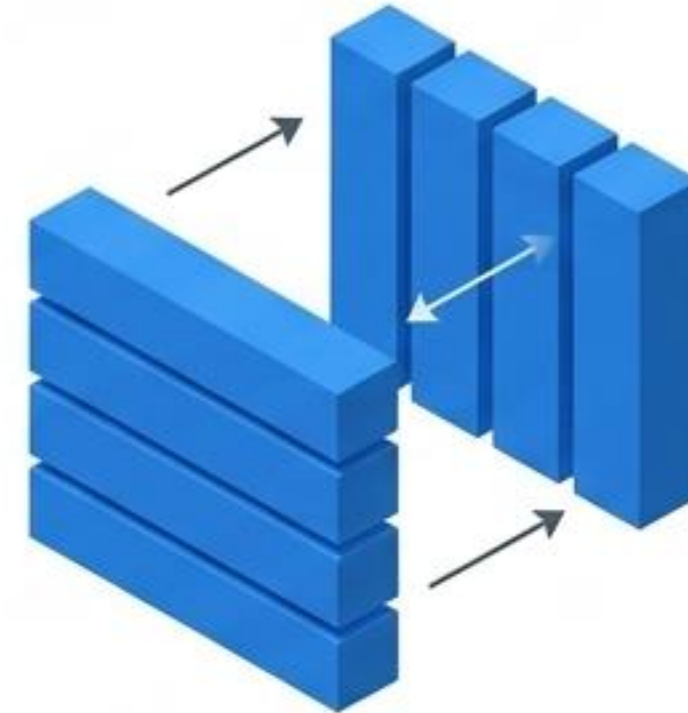
Sắp xếp lại các hàng và cột để đạt được hình dạng mục tiêu mà không thay đổi tổng số phần tử.



```
x = np.array([[0. , 1.], [2. , 3.], [4. , 5.]]) # Shape: (3, 2)
x = x.reshape ((6, 1)) # Trở thành cột 6 phần tử
```

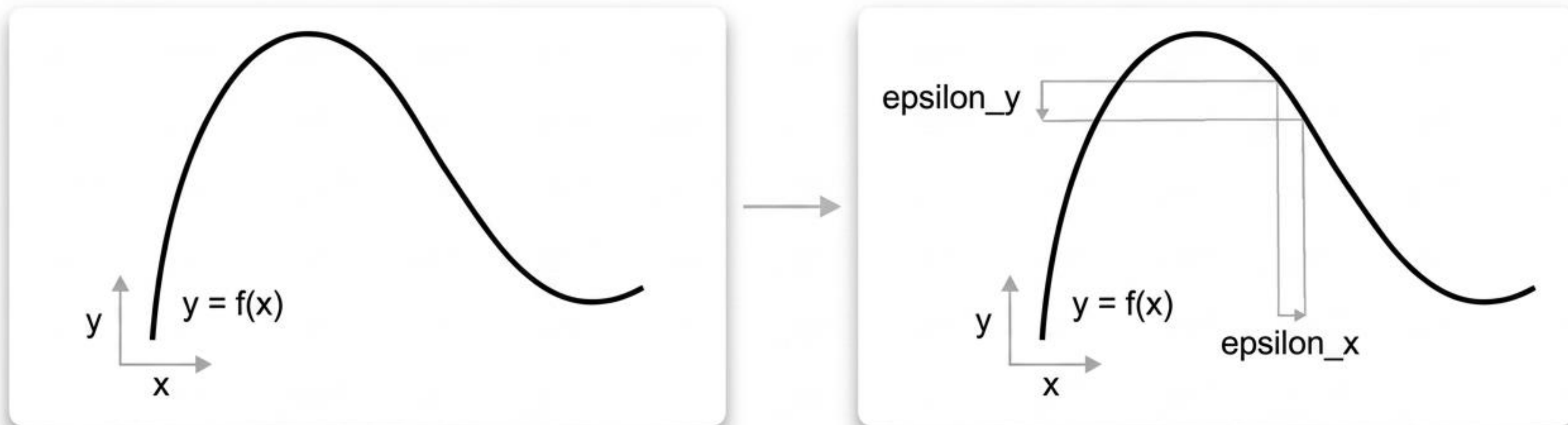
Chuyển vị (Transposing)

Tráo đổi trực tiếp hàng thành cột.



```
x = np.zeros ((300 , 20))
x = np.transpose(x) # Shape trở thành (20, 300)
```

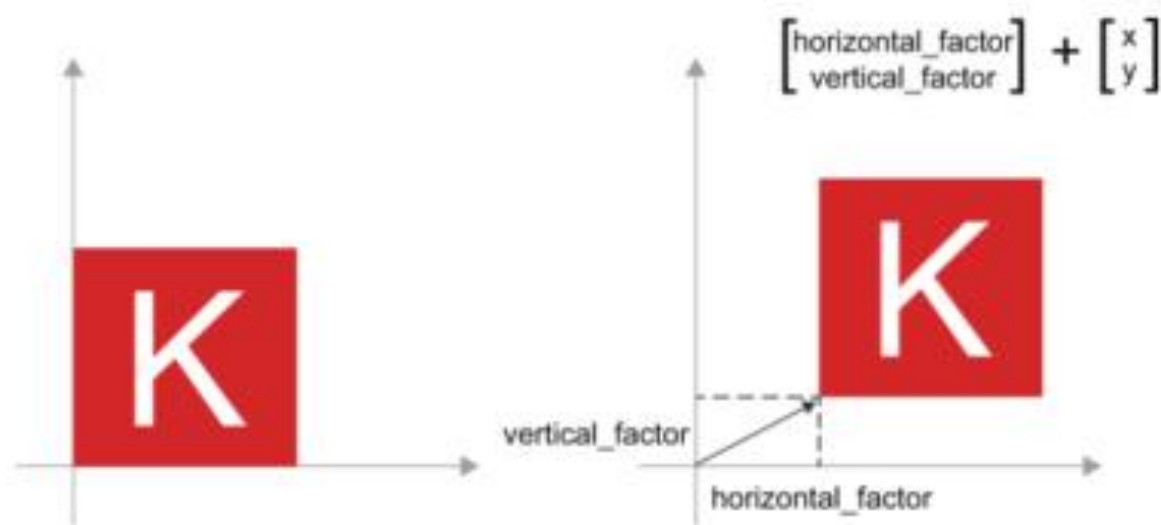

Góc nhìn Hình học: Mạng Nơ-ron là một Hàm số



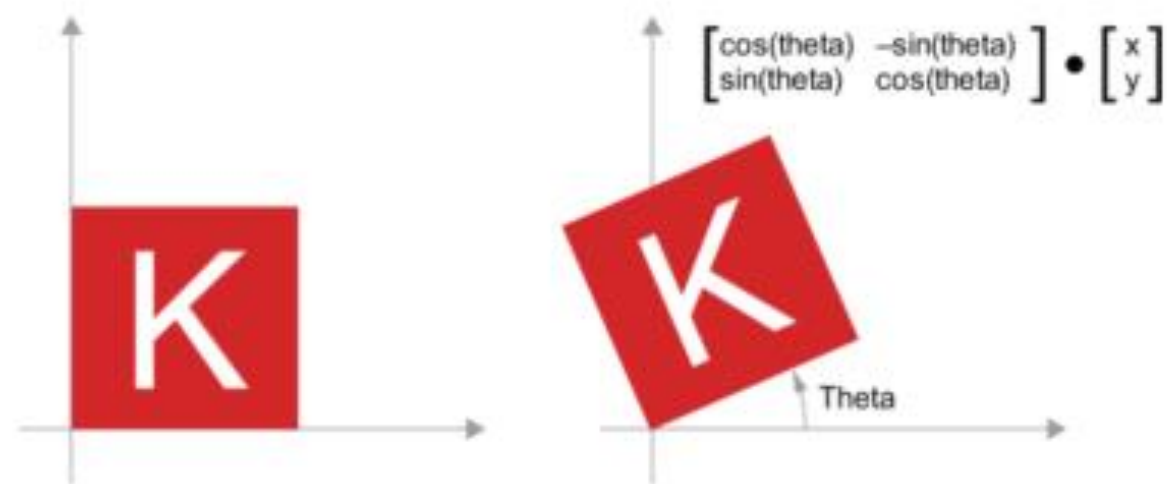
Tất cả phép toán tensor đều mang ý nghĩa biến đổi hình học.

**Bản chất của một mạng nơ-ron thực chất là một hàm toán học
khổng lồ, yêu cầu tính liên tục và trơn tru.**

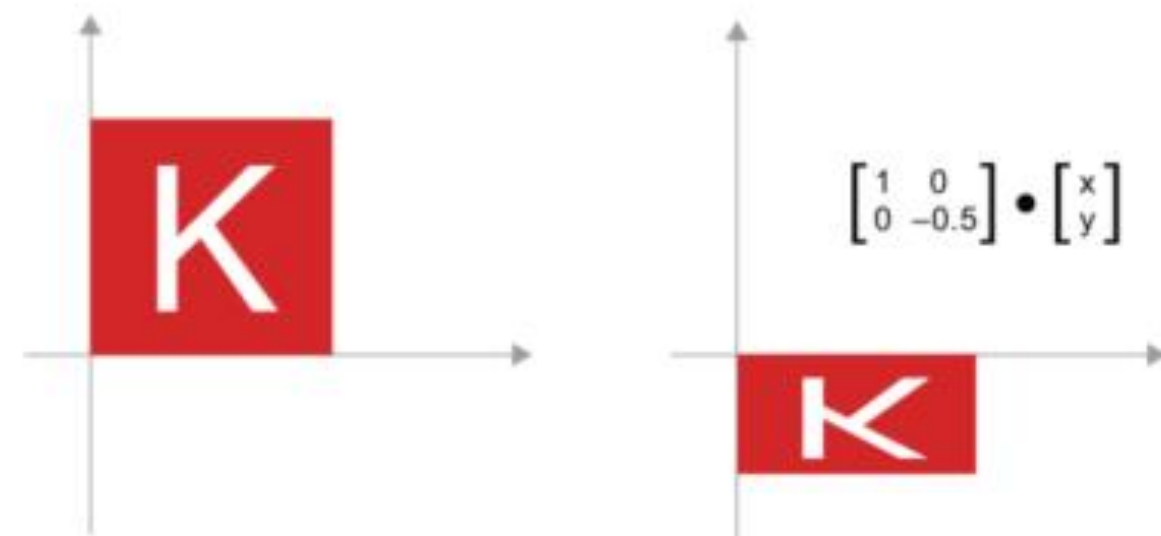
Khảo sát các Phép Biến đổi Hình học Cơ bản



Hình 2.7: **Tịnh tiến:** Dời hình bằng phép cộng vector



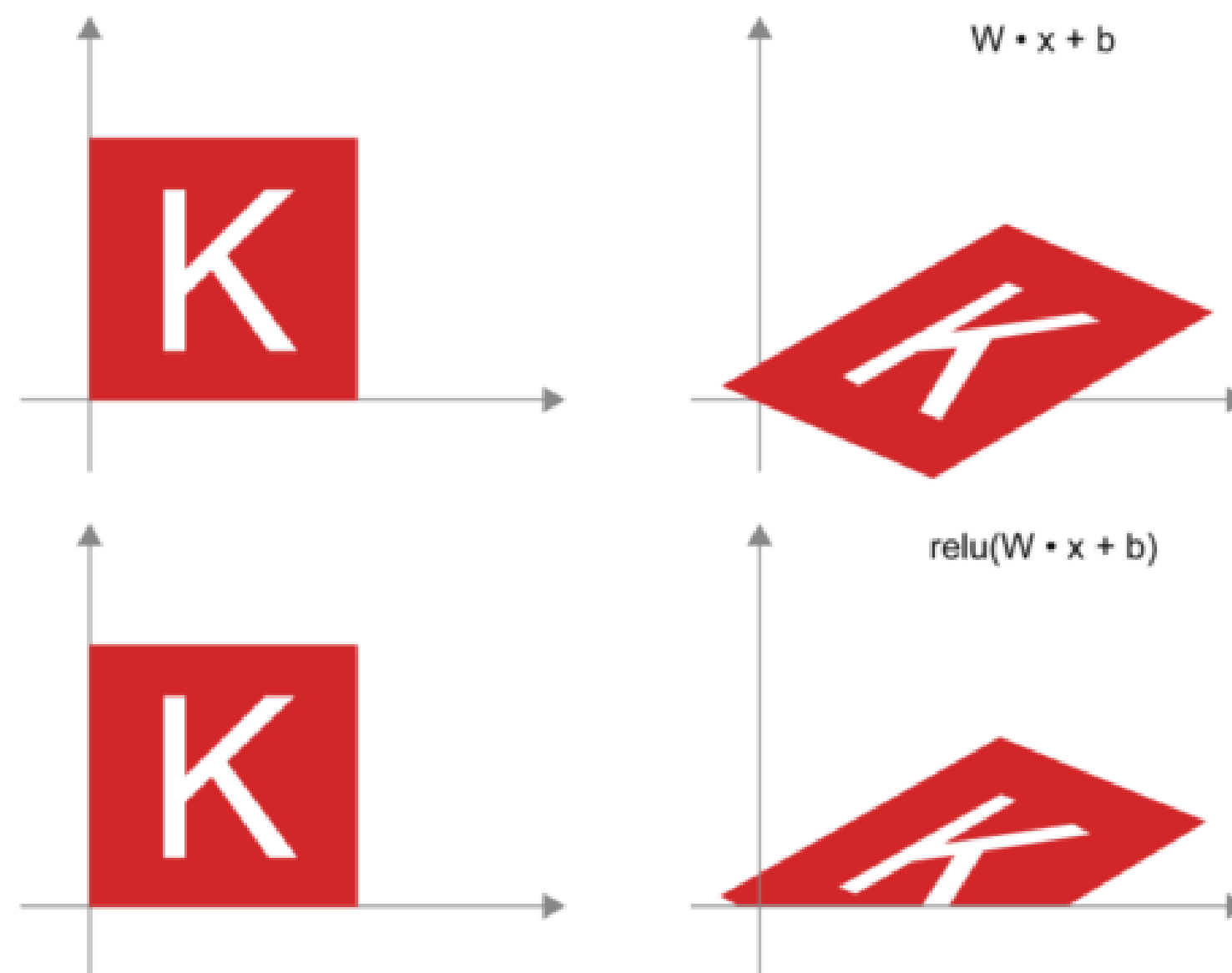
Hình 2.8: **Quay:** Tích ma trận 2×2 (Lượng giác)



Hình 2.9: **Thu phóng:** Tích ma trận đường chéo

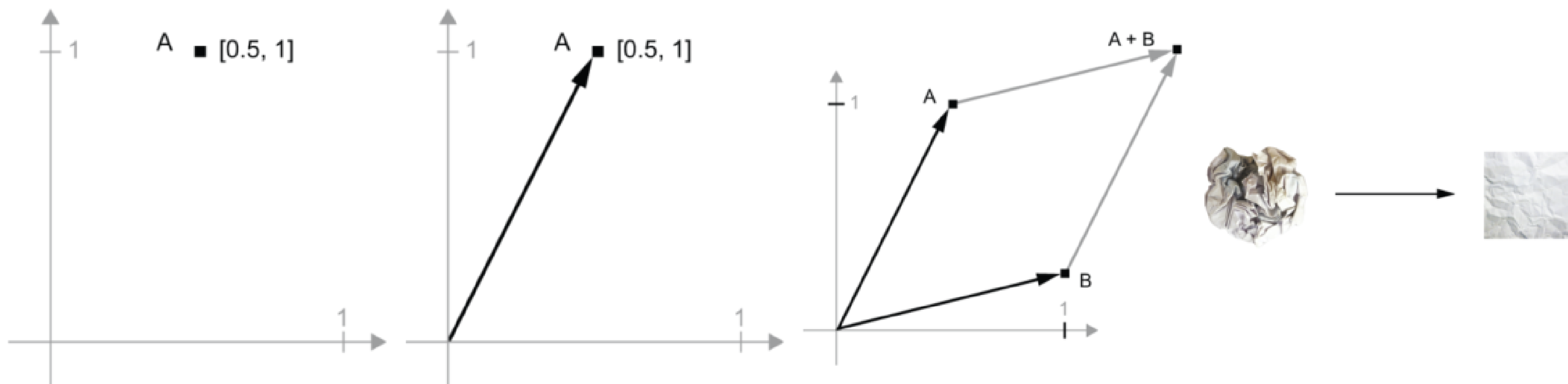
Lớp Dense & Kích hoạt Phi tuyến (ReLU)

- **Phép biến đổi Affine** là sự kết hợp của biến đổi tuyến tính (nhân W) và tịnh tiến (cộng b).
- Lớp Dense: $y = W \cdot x + b$.
- **Kích hoạt phi tuyến (ReLU)**: Giúp bóp méo không gian affine, tạo ra các ranh giới phức tạp (để phân tách dữ liệu).



Hình 2.10: Biến đổi Affine & theo sau là ReLU

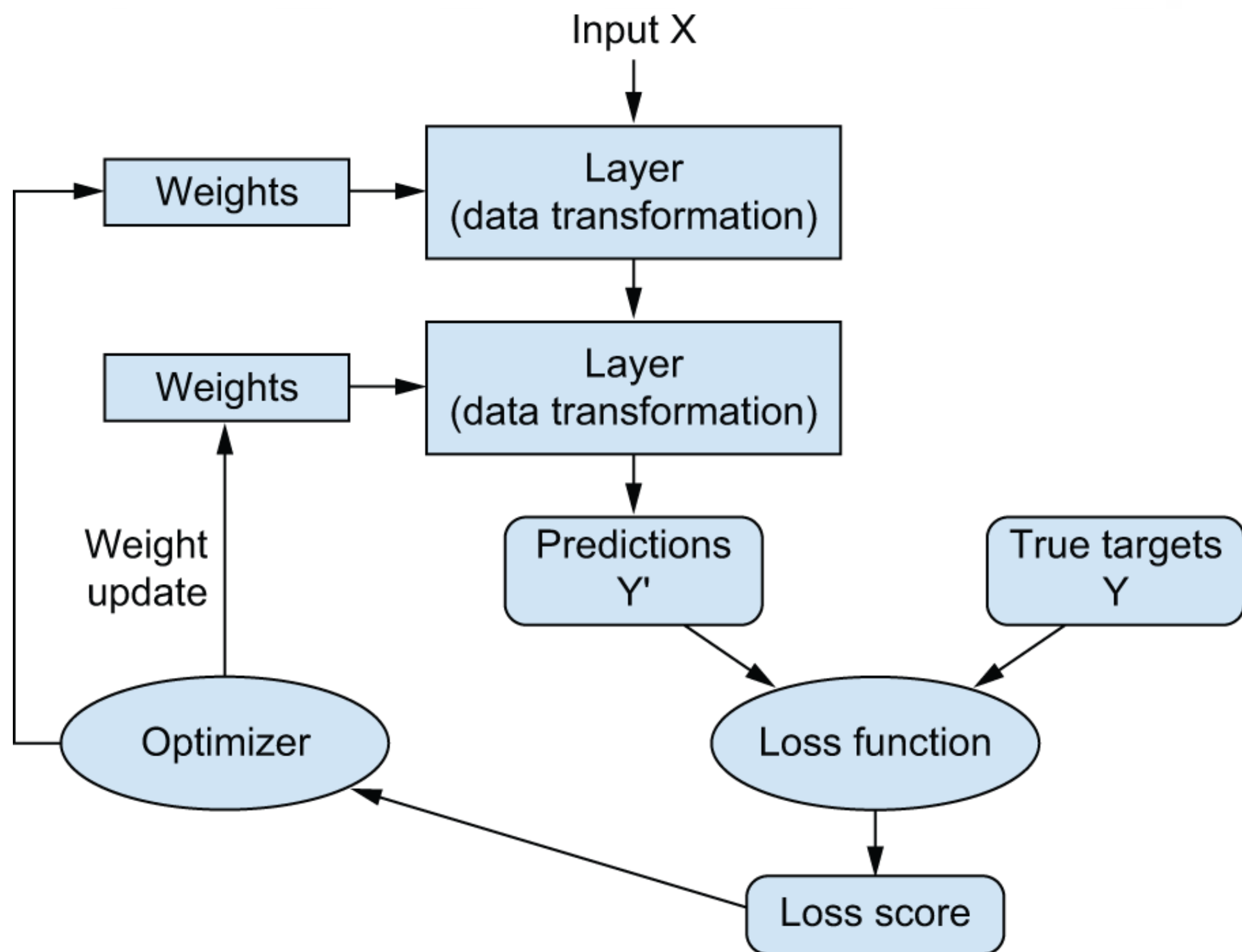
Mục đích tối thượng: Gỡ rối Dữ liệu



Hình 2.11: Quá trình phân tách dữ liệu qua các lớp ẩn giống như việc từ từ vuốt phẳng một quả bóng giấy vò nát.

**Học sâu bản chất là một chuỗi các phép biến đổi hình học tinh vi.
Mục tiêu: "Gỡ rối" (*untangle*) các đa tạp dữ liệu phức tạp đan xen nhau
thành các không gian phân tách rõ ràng.**

Hệ sinh thái Đào tạo: Vòng lặp Học (Training Loop)



Khởi tạo ngẫu nhiên trọng số mô hình W và b .

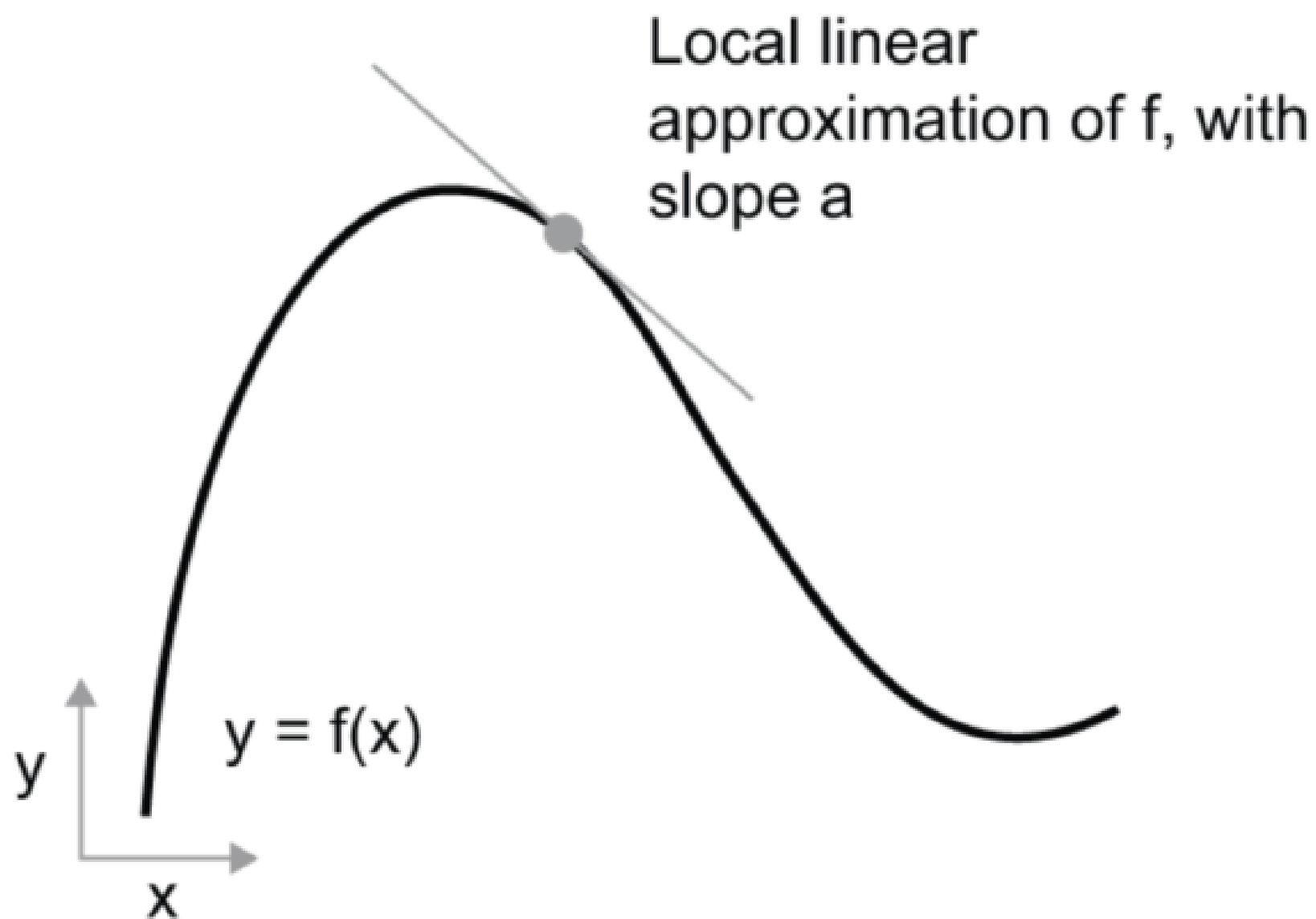
1. Lấy một lô ngẫu nhiên dữ liệu huấn luyện x và nhãn y_{true} .

2. Forward Pass: Chạy mô hình trên x để dự đoán y_{pred} .

3. Tính hàm Mất mát (Loss) để đo lường sai số dự đoán.

4. Cập nhật trọng số: Điều chỉnh tất cả trọng số theo hướng làm giảm Loss (thông qua **Gradient Descent**).

Chìa khóa Tối ưu: Khái niệm Đạo hàm (Derivative)



Hình 2.12: Đạo hàm là hệ số góc của tiếp tuyến

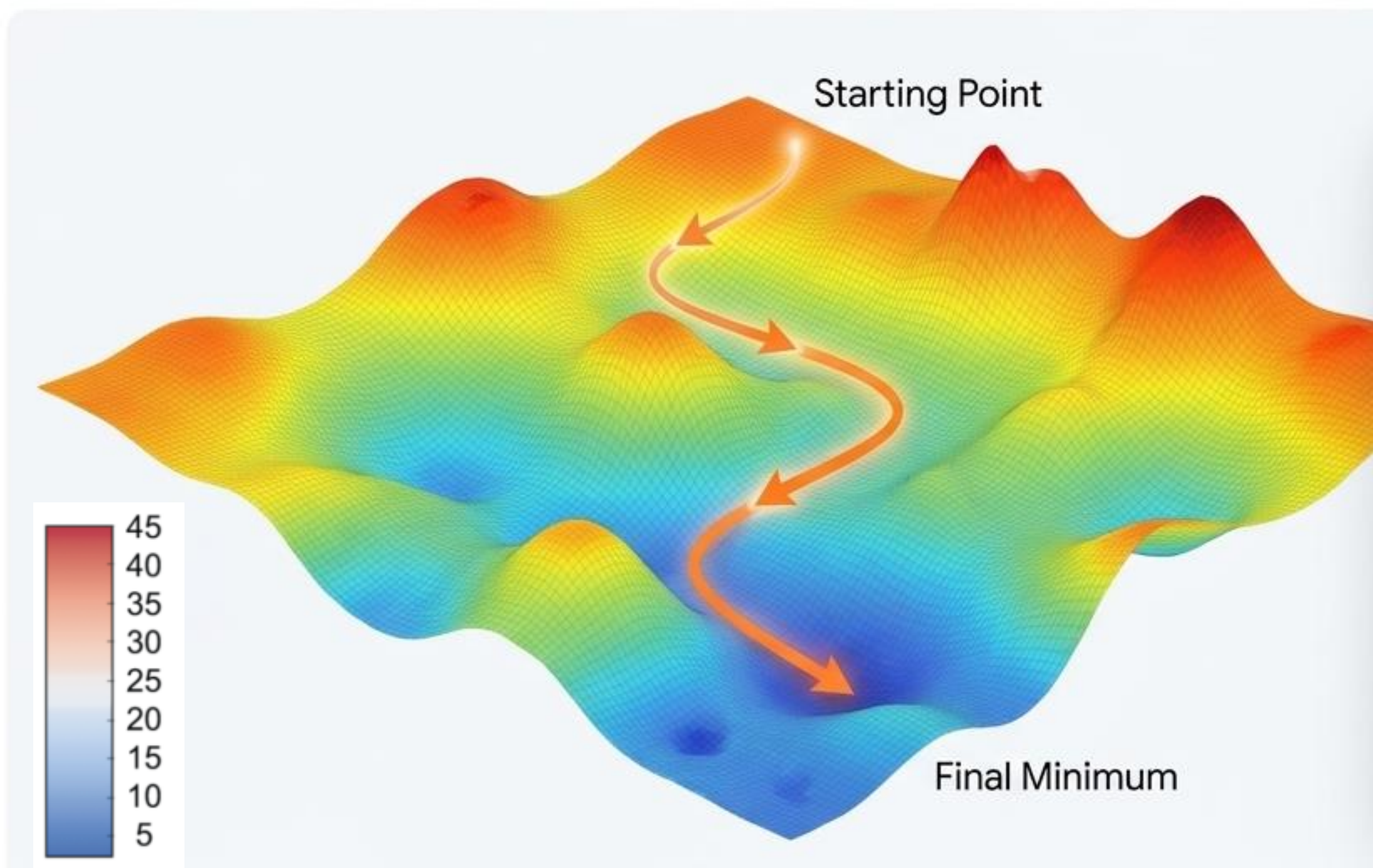
Đạo hàm $f'(x)$ thể hiện độ dốc (slope) của tiếp tuyến với hàm số $f(x)$ tại điểm x .

Quy tắc cập nhật:

- Nếu $f'(x) > 0$, tăng x sẽ làm hàm tăng.
- Nếu $f'(x) < 0$, tăng x sẽ làm hàm giảm.

Kết luận: Muốn giảm giá trị Loss, hệ thống chỉ cần đi ngược hướng với đạo hàm.

Gradient: Mặt cong Đa chiều của Loss



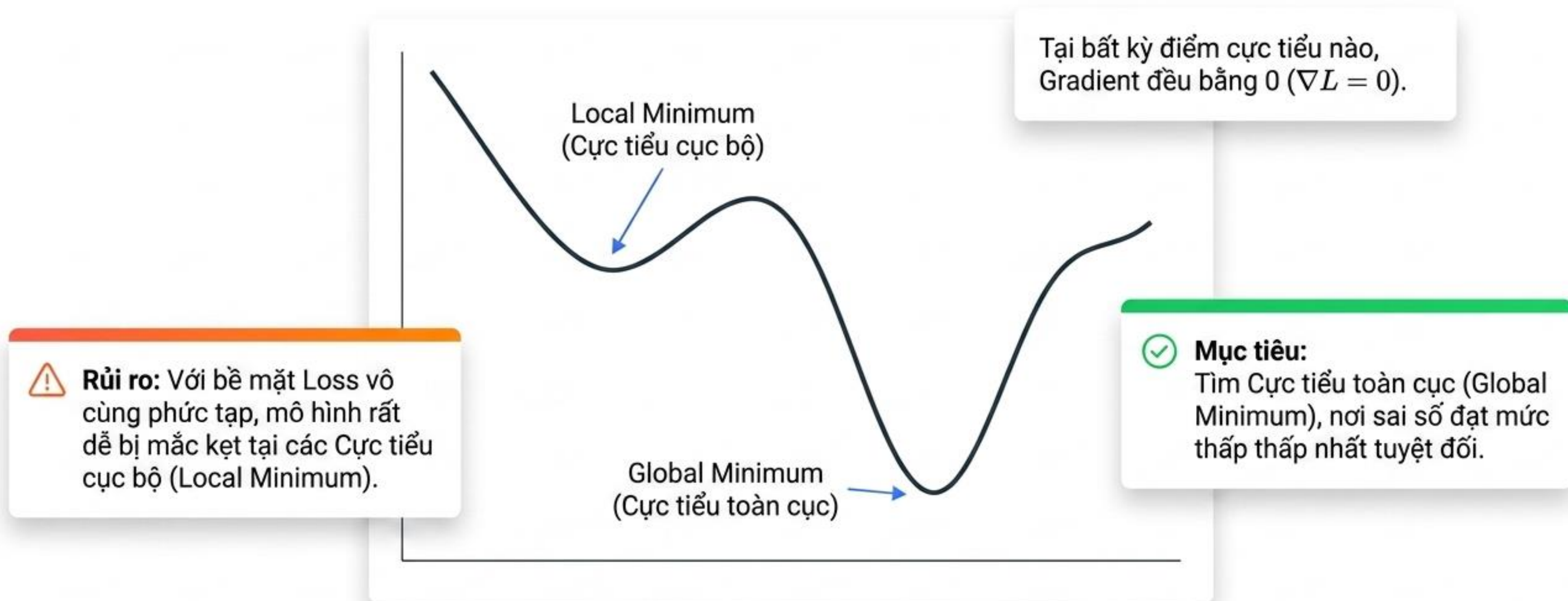
Gradient: Sự tổng quát hóa của đạo hàm trong không gian đa chiều chứa hàng triệu tham số. Nó biểu thị độ cong của bề mặt hàm Loss.

Vectơ Gradient luôn chỉ về hướng hàm L tăng nhanh nhất.

Hành trình Tối ưu: Đi ngược hướng Gradient chính là con đường ngắn nhất để trượt xuống 'thung lũng' Loss.

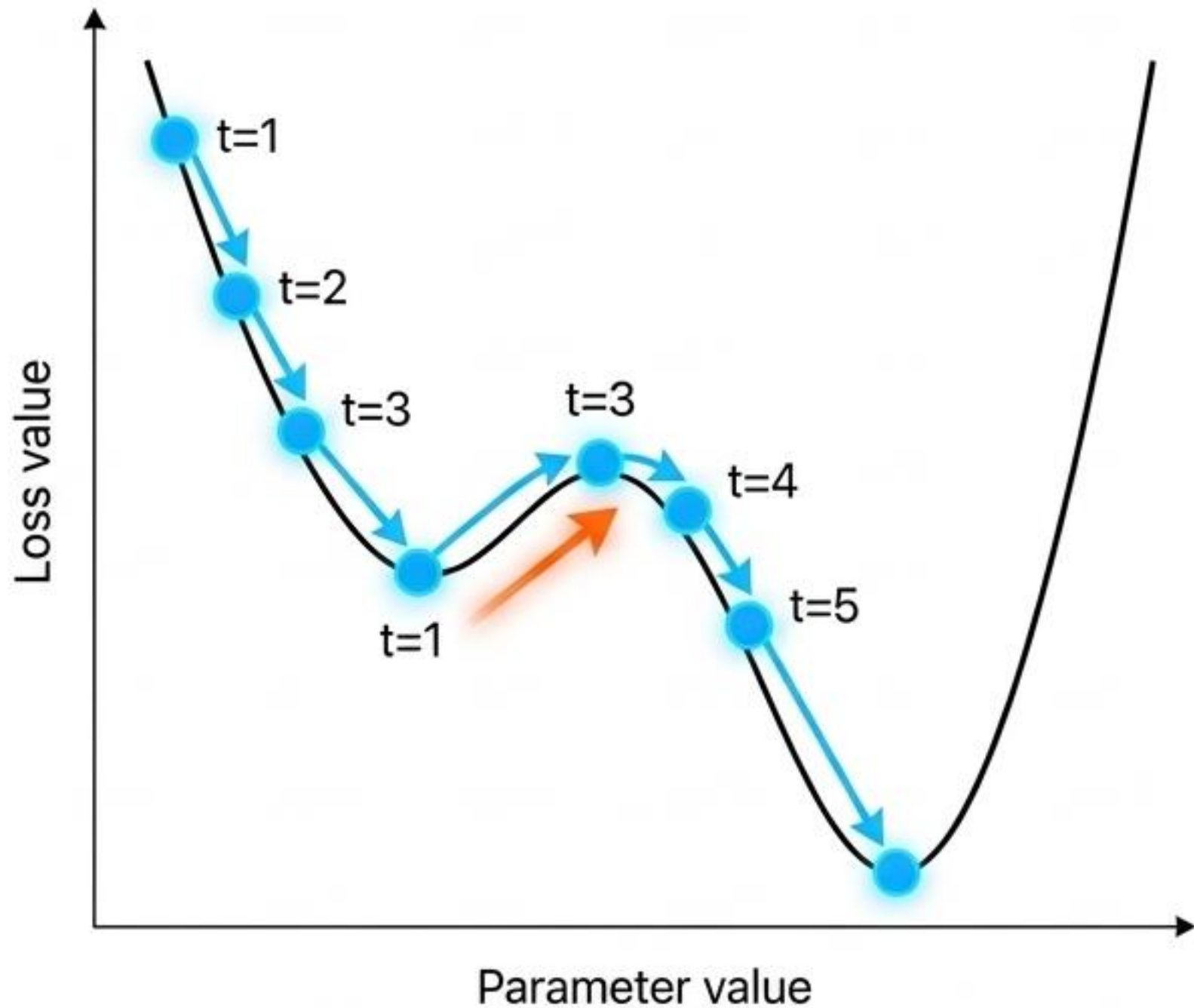
Hình 2.13: Mặt cong 3D của Loss function

Bầy Tối ưu: Cực tiểu Toàn cục và Cục bộ



Hình 2.14: Local Minimum vs Global Minimum

Thuật toán Tối ưu: SGD và Động lượng (Momentum)



Stochastic Gradient Descent (SGD)

Cập nhật trọng số dựa trên một mẻ nhỏ ngẫu nhiên.

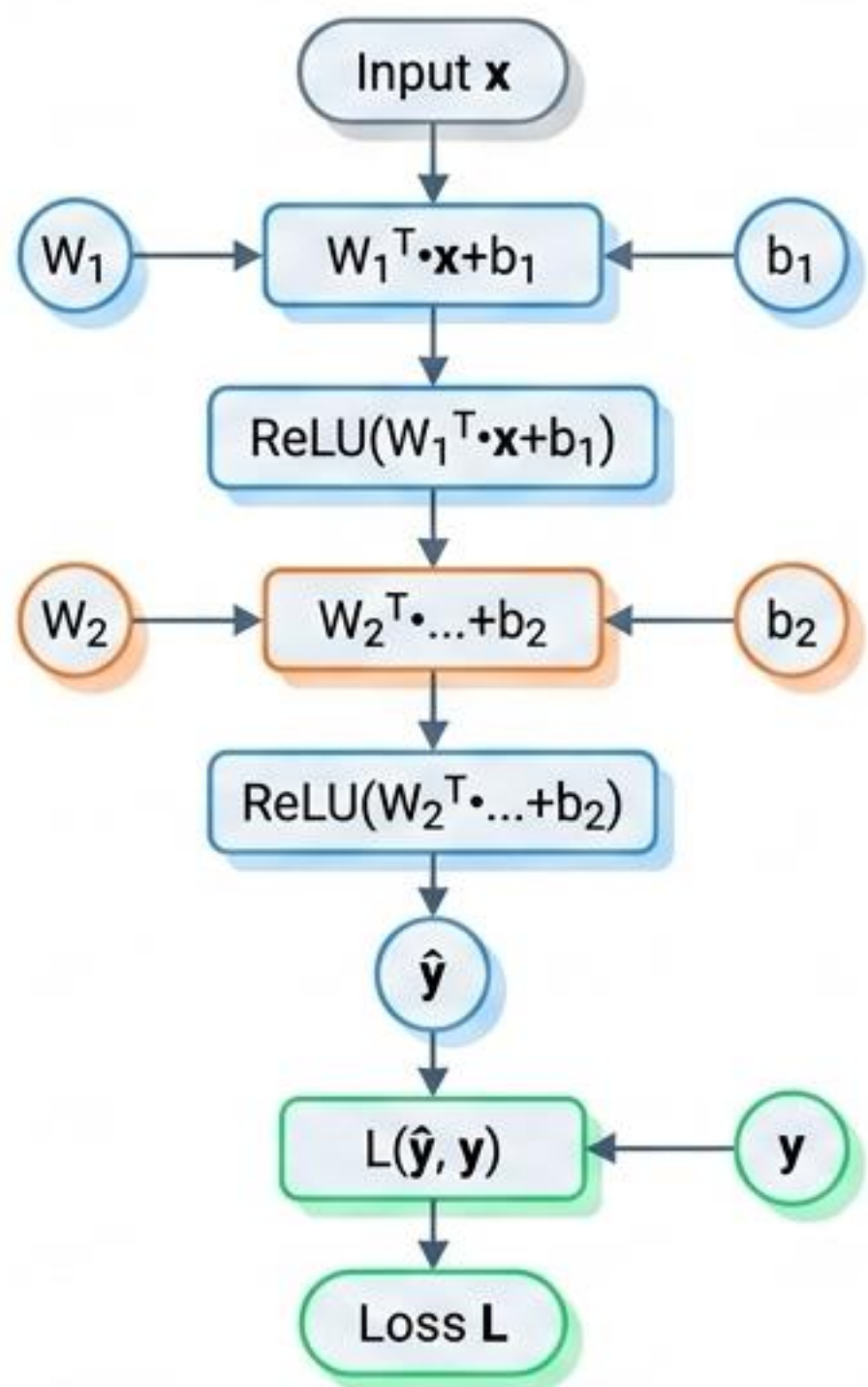
$$\text{Công thức: } W \leftarrow W - (\text{learning_rate} \times \nabla L)$$

Động lượng (Momentum)

Lấy cảm hứng từ vật lý (quả bóng lăn xuống dốc). Sử dụng vận tốc từ quá khứ để gia tốc quả bóng, giúp nó có đủ 'đà' để vọt qua và thoát khỏi các cực tiểu cục bộ hẹp.



Kiến trúc Hộp Đen: Đồ thị Tính toán (Computation Graph)



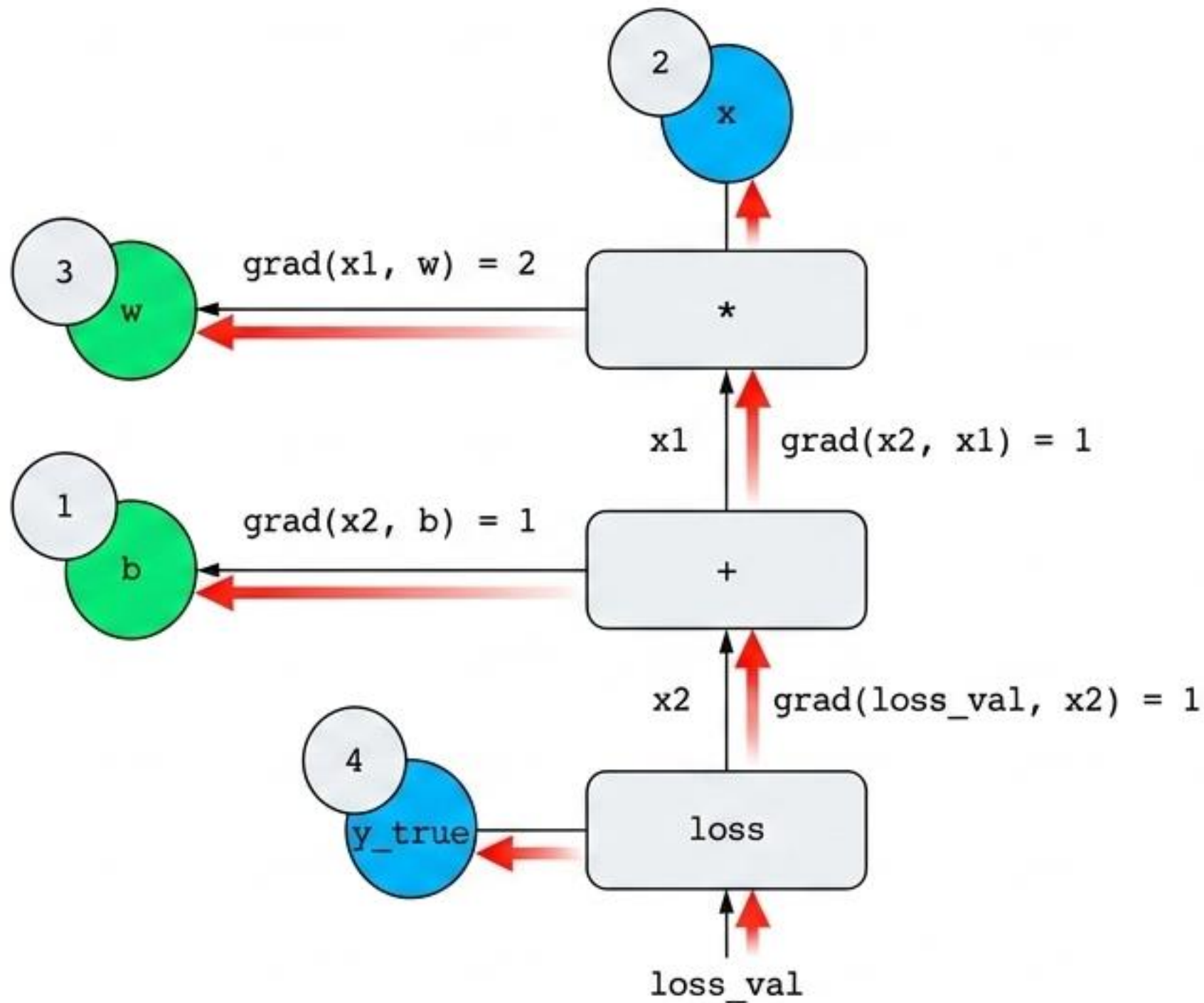
Bài toán: Làm sao để tính đạo hàm của hàng triệu trọng số cùng lúc?

Giải pháp: Cốt lõi của mô hình là Đồ thị tính toán.

Cơ chế: Biến các phép toán Tensor thành các 'nút' (node) và dòng dữ liệu thành các 'cạnh'. Cấu trúc hóa toán học thành dữ liệu máy tính.

Hình 2.16: Biểu diễn đồ thị cho mô hình mạng nơ-ron

Lan truyền ngược & Quy tắc Chuỗi (Chain Rule)



Lan truyền ngược (Backpropagation):

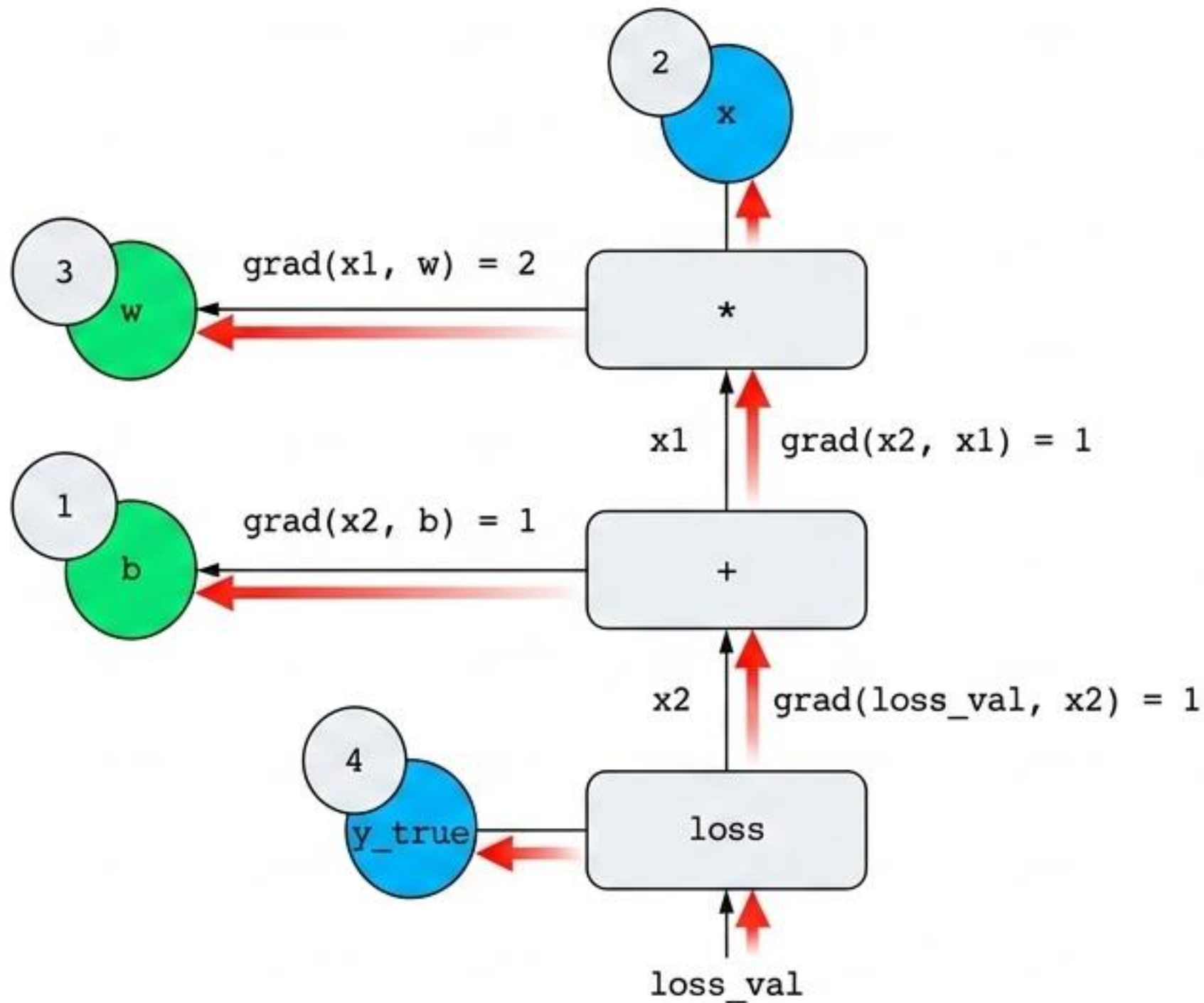
Dòng chảy thông tin đi ngược từ Loss function lên các layer đầu tiên để tính toán Gradient cho từng trọng số.

Quy tắc Chuỗi: Nền tảng toán học cho phép phân rã và tính đạo hàm của các hàm hợp phức tạp.

Công thức:

$$\frac{d(f(g(x)))}{dx} = f'(g(x)) \cdot g'(x)$$

Lan truyền ngược & Quy tắc Chuỗi (Chain Rule)



Lan truyền ngược (Backpropagation):

Dòng chảy thông tin đi ngược từ Loss function lên các layer đầu tiên để tính toán Gradient cho từng trọng số.

Quy tắc Chuỗi: Nền tảng toán học cho phép phân rã và tính đạo hàm của các hàm hợp phức tạp.

Công thức:

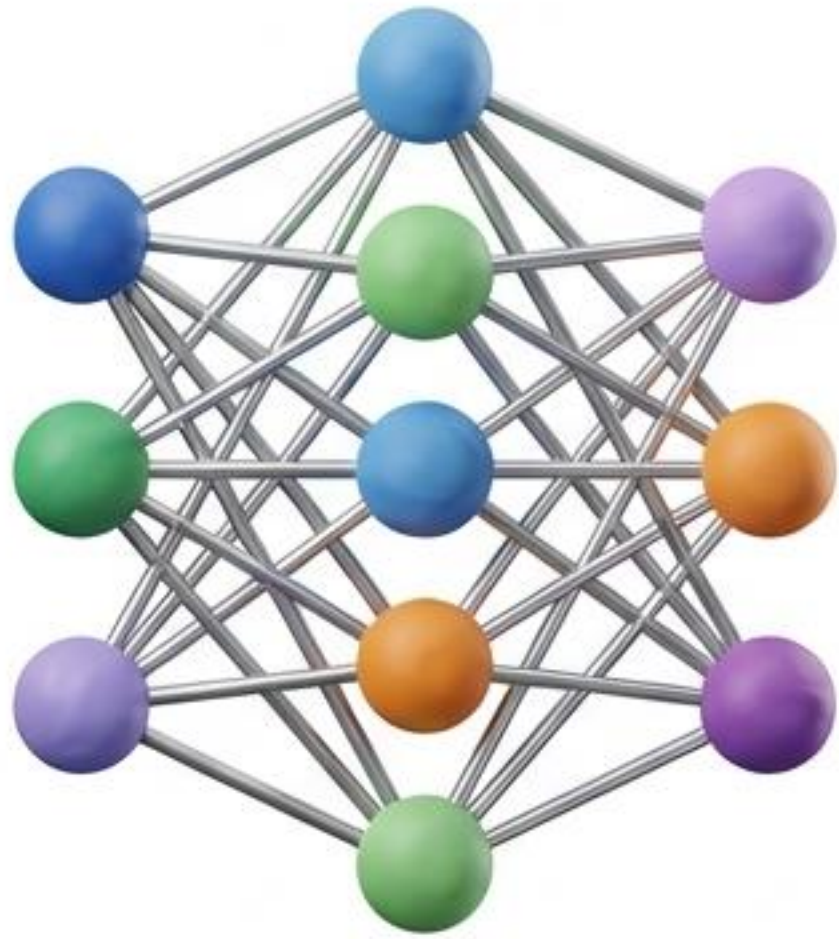
$$\frac{d(f(g(x)))}{dx} = f'(g(x)) \cdot g'(x)$$



Tự động phân biệt với TensorFlow GradientTape

Thực tế, Framework hiện đại như TensorFlow tự động hóa toàn bộ việc Lan truyền ngược bằng Automatic Differentiation.

```
1 import tensorflow as tf
2
3 x = tf.zeros(shape=()) # Gia tri khoi tạo
4 with tf.GradientTape() as tape:
5     # Tape (Bang ghi) se lưu lại do thi cac phép toan
6     y = 2 * x + 3
7
8 # Yeu cau Tape tinh gradient cua y doi voi x (dao ham)
9 grad_of_y_wrt_x = tape.gradient(y, x)
```



Thực hành (Từ đầu): NaiveDense

Hãy xây dựng thủ công một lớp Dense sử dụng tensor ops.

```
1 import keras
2 from keras import ops
3
4 class NaiveDense:
5     def __init__(self, input_size, output_size,
6         activation=None):
7         self.activation = activation
8         self.W = keras.Variable(shape=(input_size, size),
9             initializer="uniform")
10        self.b = keras.Variable(shape=(output_size, ),
11            initializer="zeros")
12
13    def __call__(self, inputs): # Thực hiện Forward Pass
14        x = ops.matmul(inputs, self.W) + self.b
15        if self.activation:
16            x = self.activation(x)
17        return x
```




Thực hành (Từ đầu): Vòng lặp Huấn luyện

Khái quát lại quá trình **model.fit()** bằng đoạn code sau:

```
1 def one_training_step(model , images_batch , labels_batch):
2     with tf.GradientTape () as tape:
3         # 1. Forward pass de du doan va tinh Loss
4         predictions = model(images_batch)
5         loss = ops.sparse_categorical_crossentropy(labels_batch , predictions)
6         average_loss = ops.mean(loss)
7
8         # 2. Backward pass (Tinh gradient tu dong nho Tape)
9         gradients = tape.gradient(average_loss , model.weights)
10
11        # 3. Cap nhat trong so bang Optimizer (nhu SGD)
12        update_weights(gradients , model.weights)
13
14    return average_loss
```


Tổng kết Chương 2



Tensors: Nền tảng cấu trúc dữ liệu của học sâu. Bao gồm các bậc 0D, 1D, 2D, 3D, 4D v.v.



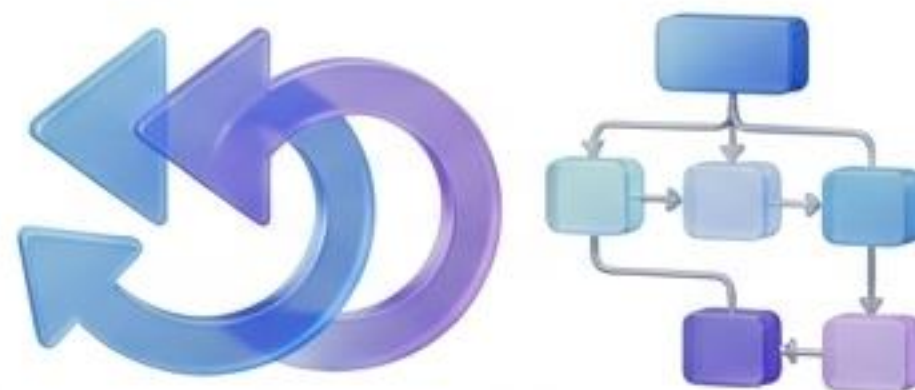
Phép toán Tensor: Phép nhân ma trận, phép cộng, ReLU... là những biến đổi hình học tinh vi giúp gỡ rối đa diện dữ liệu.



Trọng số (Weights): Tham số học hỏi chứa toàn bộ "kiến thức" của mô hình mạng.



Loss function & Optimizers: Loss đo lường độ chính xác (mục tiêu cần tối thiểu hóa). Optimizer như SGD giảm Loss thông qua đi ngược Gradient.



Backpropagation: Sử dụng Đồ thị tính toán và Quy tắc chuỗi để tính đạo hàm ngược một cách nhanh chóng qua cơ chế phân biệt tự động (GradientTape).